

08

Recuperacion de contraseña: Resend vs SMTP vs Outlook

Por que el correo no sale en Railway, que opciones existen y como quedaria el modulo de recuperacion de contraseña de este backend si se migra a una API HTTPS.

Campo	Valor
Documento	08-Recuperacion-Contraseña-Resend-SMTP-Outlook.pdf
Serie	Continua la 01, 02, 03, 04, 05, 06 y 07 de docs/
Proyecto	Compras-Backend (com.erikjarquin.compras)
Modulo afectado	Recuperacion de contraseña del login
Endpoints	POST /api/local/auth/forgot-password (y /reset-password)
Clases reales hoy	AuthController, AuthImpl, EmailService, UserEntity, UserRepository, application.yaml
Problema	Railway bloquea el SMTP saliente en planes no Pro
Alcance	Explicativo y plan. NO implementa código
Que hay que decidir	Resend solo, o Resend primario + SMTP de respaldo

Como leer este documento

Las secciones 1 a 4 explican el problema y las tres opciones. La seccion 5 describe el estado real del código de este proyecto, leído de los archivos, para que se vea que hay que tocar y que no. La seccion 6 es el plan de clases a crear y modificar, sin escribir ni una línea de implementación. Las secciones 7 a 11 son el paso a paso, las pruebas, los riesgos y el glosario.

0. Resumen en cinco lineas

1. Railway en plan Hobby **descarta los paquetes SMTP** (puertos 25, 465 y 587) por politica de plataforma. Pagar cuota de recursos no lo cambia: son dos cosas distintas.
2. El error *Connection timed out connecting to smtp.gmail.com:587* es exactamente eso: el paquete sale de la maquina pero nadie lo contesta.
3. La salida economica es **cambiar de SMTP a una API HTTPS**. Eso usa el puerto 443, que Railway no bloquea. Resend es la recomendada y tiene plan gratuito.
4. En este proyecto el cambio es **muy pequeno**: el correo solo se usa en recuperacion de contraseña y lo produce una sola clase, *EmailService*.
5. No se pueden activar los tres proveedores a la vez enviando el mismo correo: eso triplica el correo. Lo correcto es un primario y un respaldo, ambos detras de una sola interfaz.

La conclusion en una frase

Para Railway Hobby: **cambiar a Resend** (o a cualquier API HTTPS de correo). El SMTP se conserva unicamente como respaldo para desarrollo local, donde si funciona porque no hay firewall intermedio.

1. Que esta pasando en Railway (el diagnostico)

1.1 No es un problema tecnico de Spring

El backend ya esta bien configurado. *application.yaml* tiene el bloque de correo y Spring Boot arma el cliente SMTP solo. El problema esta antes de Spring: en la red.

1.2 Los dos limites que se estan confundiendo

Limite	Que controla	Como se sube	Aplica aqui
Cuota de recursos	CPU, RAM, disco, ancho de banda	Pagar un plan o un credito de consumo	Si : por eso el plan ya no se traba
Restriccion de red	Que puertos puede abrir la maquina	Plan Pro o superior, y despues rededploy	Si : esta es la que bloquea el correo

Por que pagar no lo arreglo

La cuota de recursos y la politica de red son controles separados. Comprar credito de consumo sube la primera. La segunda solo sube con el plan Pro, y **ademas exige un redeploy**: cambiar el plan no cambia la maquina actual, hay que volver a construir el despliegue para que la politica nueva aplique.

1.3 Que se ve desde el log

```
org.springframework.mail.MailSendException:
  Failed to create a connection to the SMTP server
  Connection timed out: connect
  -> smtp.gmail.com:587 (o smtp.office365.com:587)
```

Un *timeout* y no un *connection refused* es la firma del firewall: un puerto cerrado de verdad se rechaza de inmediato (refused), mientras que un paquete descartado se queda esperando hasta que expira el tiempo. Esa diferencia es lo que confirma que hay un filtro de por medio y no un mal usuario o contraseña.

1.4 Los puertos afectados

Puerto	Uso	Estado en Railway Hobby
25	SMTP sin cifrar	Bloqueado
587	SMTP con STARTTLS (el que se usa hoy)	Bloqueado
465	SMTP sobre TLS implicito	Bloqueado
443	HTTPS (API de correo, web, REST)	Permitido

La consecuencia de diseno

Todo lo que se haga con una API HTTPS de correo sale por 443 y no lo nota. Por eso la solucion no es 'arreglar el SMTP' sino **cambiar de transporte**. El contenido del correo (asunto, cuerpo, enlace) no cambia en absoluto: lo unico que cambia es como se entrega.

2. Las tres opciones, cara a cara

2.1 Tabla general

Criterio	Gmail SMTP	Outlook / M365 SMTP	Resend (API HTTPS)
Transporte	SMTP 587 + STARTTLS	SMTP 587 + STARTTLS	POST HTTPS a api.resend.com (443)
Como se autentica	Usuario + App Password (exige 2FA)	Usuario + App Password u OAuth2 segun el tenant	API key como cabecera Authorization: Bearer
Funciona en Railway Hobby	NO	NO (y M365 suele bloquear la autenticacion basica)	SI
Funciona en local (tu PC)	SI	SI	SI
Cuenta de prueba	Mail de prueba dedicado	Cuenta de pruebas de M365	onboarding@resend.dev (solo a tu correo)
Costo	Gratis con limites diarios	Gratis con limites diarios	Gratis hasta ~3000 correos/mes, luego por uso
Remitente con dominio propio	SI (alias o cuenta completa)	SI (si el tenant lo permite)	SI, verificando el dominio con DNS
Cifrado del transito	STARTTLS	STARTTLS	HTTPS (TLS)
Cuerpo en HTML	SI (JavaMailSender + MimeMessage)	SI (JavaMailSender + MimeMessage)	Nativo: campo <i>html</i>
Idempotencia de envio	No	No	SI : cabecera <i>Idempotency-Key</i>
Reintentos y webhooks	No	No	SI (reintentos y eventos)
Dependencia del proveedor	Cuenta de Google	Licencia o tenant de M365	Una API key; el dominio sigue siendo tuyo
Riesgo de bloqueo del remitente	Alto desde un servidor nuevo	Alto, con MFA adicional que no siempre aplica	Bajo: pasa por HTTPS, igual que un webhook

2.2 Que es exactamente una API HTTPS de correo

La idea es quitar el protocolo SMTP de en medio. En vez de abrir un socket y hablar con un servidor de correo, se hace una peticion HTTP POST a una direccion como <https://api.resend.com/emails>, con un JSON en el cuerpo y una API key en la cabecera.

Para el programador la diferencia es pequena: se sigue '**enviar un correo a una direccion con un asunto y un texto**'. Lo que cambia por dentro es que la peticion es una llamada HTTP ordinaria, y por eso el puerto deja de ser 587 y pasa a ser 443.

2.3 Forma de la llamada (ilustrativo, no es codigo del proyecto)

```
POST https://api.resend.com/emails
Authorization: Bearer re_XXXXXXXXXXXXXXXXX
Content-Type: application/json
```

```
{
  "from": "no-reply@tudominio.com",
  "to": ["persona@ejemplo.com"],
  "subject": "Recuperacion de contraseña",
  "text": "Da clic en el siguiente enlace:\n\n<el enlace>"
}
```

Un detalle que hace perder horas

El campo *to* es una **lista**, no un texto: *"to": ["..."]*. Si se manda como cadena suelta, la API responde 422 y el correo nunca sale. Ademas *from* solo se acepta si el dominio esta verificado en Resend; la unica excepcion es *onboarding@resend.dev*, que ademas **solo envia al correo con el que te registraste**. Todo lo demas cae en la carpeta de spam.

3. Se pueden configurar los tres al mismo tiempo?

3.1 Respuesta corta

Si se pueden tener los tres configurados (credenciales, propiedades, variables de entorno). **No se pueden tener los tres enviando a la vez**: el usuario recibiría tres correos identicos por cada recuperacion de contrasena.

3.2 Las tres formas de combinarlos

Combinacion	Como funciona	Pros	Contras	Veredicto
Solo uno, elegido por entorno	<code>app.mail.provider</code> vale resend en produccion y smtp en local. En cada arranque existe un solo bean.	Simple, sin logica de reintentos, facil de depurar	Si ese proveedor falla, no hay correo	Recomendada para empezar
Primario + respaldo	Se intenta el primario; si falla, se intenta el respaldo y se registra el fallo.	Tolera caidas de uno de los dos	Un poco mas de codigo; hay que evitar el envio doble	Recomendada a medio plazo
Los tres a la vez	Tres beans activos que disparan los tres transportes	-	Triple correo, reputacion danada, coste por tres	Nunca

La trampa del envio doble en el modo con respaldo

El error clasico: se manda con el primario, la excepcion se lanza *despues* de que el servidor ya recibio el correo (por ejemplo, un timeout de respuesta cuando el correo ya salio), cae el respaldo y el usuario recibe dos. Por eso el respaldo debe dispararse solo ante fallos **claramente previos** al envio (credenciales invalidas, dominio no verificado, error 4xx de la API), no ante cualquier excepcion. En la practica, para este modulo, lo mas simple es que la excepcion se registre y el flujo de recuperacion siga igual: que un correo no salga no debe romper la respuesta del endpoint.

3.3 Que sigue igual y que cambia

- **Sigue igual**: el enlace, el asunto, el texto, el token, la vigencia de una hora, el endpoint, la respuesta HTTP y la pantalla del frontend.
- **Cambia**: solo la clase que se encarga de entregar el mensaje.
- **No debe cambiar**: la firma que usa el modulo de autenticacion. Ese contrato es la frontera del cambio, y conviene respetarlo para que el resto del proyecto no se entere de nada.

4. Como se configura cada opcion

4.1 Gmail por SMTP (como esta hoy)

1. Activar la verificacion en dos pasos en la cuenta de Google.
2. Generar una **contraseña de aplicacion**: Google da una clave de 16 caracteres que es lo unico que sirve para SMTP.
3. Anotar los cuatro parametros que el proyecto ya tiene en *application.yaml* (ver seccion 5.3).
4. Probar en local contra el servidor de correo.
5. **En Railway Hobby no hay paso 5**: el puerto esta bloqueado. Solo funciona en plan Pro, y con redeploy.

Por que hace falta la contraseña de aplicacion

Google dejo de aceptar la contraseña normal de la cuenta en SMTP. Sin 2FA activado no hay ninguna forma de enviar por SMTP; con 2FA activo, la clave de la cuenta queda inutilizada para esto. Por eso existe la contraseña de aplicacion: se puede revocar por separado, sin tocar el acceso a Gmail.

4.2 Outlook / Hotmail / Microsoft 365 por SMTP

1. Tener una cuenta de Outlook.com o de Microsoft 365.
2. Intentar una **contraseña de aplicacion** o, segun el tenant, activar **OAuth2** con el permiso *Mail.Send*.
3. Parametros: host *smtp.office365.com*, puerto 587, STARTTLS.
4. En cuentas personales de Outlook suele funcionar la contraseña de aplicacion; en cuentas de empresa depende de que el administrador del tenant la habilite.

El detalle que hace perder tiempo con Microsoft

Una cuenta de empresa puede tener desactivada por politica la autentificacion basica y el acceso de aplicaciones heredadas. El sintoma ya no es un timeout (eso seria el firewall): es un **401 con un mensaje de politica** que dice que hay que usar OAuth2. Ahi el problema no es la red, es la cuenta, y la solucion pasa por registrar una aplicacion en Entra ID, que es bastante mas trabajo que cambiar a una API HTTPS.

4.3 Resend por API HTTPS (la ruta recomendada)

1. Crear cuenta en *resend.com* con el correo que controlas.
2. Crear una **API key** en el panel. Se muestra una sola vez: guardala.
3. **Probar sin dominio**: usar *from = onboarding@resend.dev*. Solo podra enviarse al correo con el que se creo la cuenta. Sirve para comprobar que todo funciona antes de tocar DNS.
4. **Preparar el dominio**: anadir el dominio propio en el panel de Resend y publicar los registros DNS que indica (SPF, DKIM y opcionalmente DMARC).
5. Usar *from = no-reply@tudominio.com*.
6. Poner *RESEND_API_KEY* y *RESEND_FROM* como variables de entorno en el despliegue.
7. Hacer la prueba contra */api/local/auth/forgot-password*.

Por que verificar el dominio importa tanto

Un remitente sin verificar es exactamente lo que los filtros de spam distinguen de un envio fraudulento. Sin verificar, el correo sale pero aterriza en spam o en la carpeta de publicidad. Ademas, con dominio propio, si algun dia se cambia de proveedor el remitente sigue siendo el mismo y la reputacion no se pierde. Esta es una de las razones por las que **no conviene usar una cuenta personal de Gmail como remitente** de un producto: la reputacion es del dominio, y el dominio propio es la unica forma de acumularla.

5. Como esta hoy el modulo en este proyecto

Lo que sigue se leyo de los archivos del repositorio, no de memoria. Sirve para saber exactamente que hay que tocar y que no.

5.1 El recorrido completo del correo de recuperacion

```

Cliente (Angular)
  | POST /api/local/auth/forgot-password { email }
  v
AuthController      (controller/AuthController.java)
  v
AuthService / AuthImpl (service/impl/AuthImpl.java)
  1. busca el usuario por correo; si no existe, lanza UserException
  2. genera token = UUID.randomUUID()
  3. resetTokenExpiration = ahora + 1 hora
  4. guarda el usuario
  5. arma link = frontendUrl + "/reset-password?token=" + token
  6. emailService.sendPasswordRecoveryEmail(email, link)
  v
EmailService        (service/EmailService.java)
  SimpleMailMessage -> JavaMailSender -> SMTP -> internet

```

El paso 6 es el **unico punto de contacto con el correo**. Ese es el dato que hace que el cambio sea pequeno.

5.2 La clase que produce el correo

Hoy *EmailService* **no es una interfaz**: es una clase concreta anotada con *@Service* que depende directamente de *JavaMailSender*, arma un *SimpleMailMessage* (texto plano, sin HTML) y llama a *send*. El remitente lo toma de *spring.mail.username*, o sea de la misma cuenta que envia.

Dato relevante para el diseno

Que la clase sea concreta y no una interfaz significa que **no hay punto de extension**: hoy no se podria cambiar el transporte sin tocar el codigo que la usa. Crear la interfaz es justamente el primer paso del plan de la seccion 6, y es lo que permitira tener SMTP y HTTPS a la vez sin duplicar codigo.

5.3 La configuracion actual (application.yaml)

```

spring:
  mail:
    host: ${MAIL_HOST:smtp.gmail.com}
    port: ${MAIL_PORT:587}
    username: ${MAIL_USERNAME}      <- sin default
    password: ${MAIL_PASSWORD}     <- sin default
    properties:
      mail:
        smtp:
          starttls:
            enable: true

app:
  frontend-url: ${FRONTEND_URL:http://localhost:4200}

```

La trampa que hace fallar el arranque

MAIL_USERNAME y *MAIL_PASSWORD* estan escritos **sin valor por defecto**. Si se pasa a Resend y se quitan esas variables del despliegue sin tocar este bloque, la aplicacion **no levanta**: Spring no encuentra el placeholder y falla al iniciar, antes de ejecutar logica de correo alguna. Por eso el plan de la seccion 6 no borra el bloque SMTP a ciegas: o se deja completo, o se cambia a valores por defecto vacios.

5.4 Lo que NO hay que tocar

- *AuthController*: las rutas y los contratos publicos no cambian.
- *UserEntity* y *UserRepository*: el token, su vigencia de una hora y su limpieza ya son correctos y son ortogonales al transporte del correo.
- *config/security/*: la recuperacion de contraseña es publica por diseno (quien no esta autenticado no puede pedir un enlace) y eso no cambia.

- El frontend: sigue recibiendo la misma respuesta y sigue abriendo la misma pantalla *reset-password*.

6. Que clases se crearian y modificarian

Este documento no implementa nada

Lo que sigue es el **plan**: que archivos habria que crear, que habria que tocar, por que, y en que orden. No se escribio codigo en el repositorio ni se modifico ninguna clase de la aplicacion.

6.1 Las dos estrategias posibles

Criterio	Estrategia A: sustituir	Estrategia B: interfaz + proveedor
Que se hace	Se reescribe el interior de <i>EmailService</i> para que use HTTP en vez de SMTP	Se crea una interfaz, se mueven los dos transportes a clases separadas y se elige una con configuracion
Clases nuevas	1 o 2 (cliente HTTP + DTOs)	5 o 6 (interfaz, 2 implementaciones, DTOs, excepcion)
Volver a SMTP	Hay que deshacer los cambios	Solo cambiar una variable
Pruebas	Mas faciles: un solo camino	Mas trabajo: una prueba por transporte
Riesgo	Se mezcla el enlace con el transporte en una sola clase	Se puede cambiar de proveedor sin volver a tocar nada

Cual elegir

La **estrategia B** es la correcta para este proyecto, por una razon que ya se muestra en otras partes del codigo: el proyecto tiene un historial de decisiones pensadas para que la base de datos y la aplicacion no se contradigan (el costo congelado, la idempotencia de la venta, los permisos sembrados en la migracion y no solo en el arranque). Una decision de arquitectura que se toma a la ligera es la que despues alguien respeta sin entenderla. Ademas, la interfaz permite probar el envio sin red, cosa que la estrategia A no facilita.

6.2 Archivos a CREAR

#	Archivo	Responsabilidad	Por que hace falta
1	<i>service/EmailSender.java</i>	Interfaz con el metodo que hoy tiene <i>EmailService</i> : enviar el correo de recuperacion a un destinatario con un enlace	Es la frontera del cambio. Gracias a ella <i>AuthImpl</i> no sabe ni le importa si el correo sale por SMTP o por HTTPS
2	<i>service/impl/ResendEmailSender.java</i>	Implementacion que arma el JSON y hace la peticion POST al endpoint de correos con el cliente HTTP de Spring	El camino nuevo, el que desbloquea Railway
3	<i>service/impl/SmtpEmailSender.java</i>	Implementacion con el codigo actual de <i>EmailService</i> (JavaMailSender y SimpleMailMessage)	Sirve para desarrollo local y como respaldo. Sin esto, en local se pierde la opcion de probar sin depender de un tercero
4	<i>model/dto/resend/ResendEmailRequest.java</i>	Objeto con los campos que espera la API (remitente, lista de destinatarios, asunto, cuerpo)	Serializar el JSON a mano con mapas es fragil; un objeto tipado avisa en tiempo de compilacion
5	<i>model/dto/resend/ResendEmailResponse.java</i>	Objeto con el identificador del envio que devuelve la API	Sirve para registrar en el log que correo salio y con que id, util para depurar entregas
6	<i>exceptions/EmailDeliveryException.java</i>	Excepcion propia de entrega, con su codigo de error	Distinguir 'no se pudo enviar el correo' de otros fallos, y poder responder con un mensaje util en vez de un 500 generico

6.3 Archivos a MODIFICAR

#	Archivo	Que cambiar	Cuidado
1	<i>service/EmailService.java</i>	Dejar de depender de JavaMailSender. Pasa a delegar en la interfaz <i>EmailSender</i> , o se elimina y se actualiza quien lo usa	No cambiar el nombre ni los parametros del metodo que recibe destinatario y enlace: ese contrato lo consume la logica de autentificacion

#	Archivo	Que cambiar	Cuidado
2	<code>service/impl/AuthImpl.java</code>	Solo el tipo inyectado y su import. La llamada de envio, el token, la vigencia y el enlace quedan igual	Es el cambio mas pequeno de todos. Si aqui hay mas de un cambio, es que se esta haciendo el trabajo en el sitio equivocado
3	<code>application.yaml</code>	Anadir el bloque de Resend (clave, remitente, direccion de la API) y el selector de proveedor	No borrar el bloque SMTP sin sustituirlo por un equivalente: sus variables no tienen default y quitarlas sin tocar aqui impide que la aplicacion arranque
4	<code>application-local.yaml</code>	Anadir las variables de Resend para poder probar en local	Este archivo no esta en git: hay que regenerarlo en cada maquina, y ya se ha avisado de ello en este proyecto
5	<code>pom.xml</code>	Casi nada. El cliente HTTP que se use ya viene con el starter de web	Unico aviso real: si se elige el cliente reactivo de Spring hay que anadir su starter. Con el cliente clasico, ninguna dependencia nueva
6	<code>exceptions/GlobalExceptionHandler.java</code>	Anadir el manejador de la excepcion de entrega, si se quiere una respuesta limpia	Que el fallo de correo no se traduzca en una pantalla de error del servidor

6.4 Archivos que NO se tocan

- `controller/AuthController.java`: rutas, permisos y contratos publicos.
- `model/entity/UserEntity.java`: el token, su vigencia y su limpieza.
- `repository/UserRepository.java`: la busqueda del token y del correo.
- `config/security/`: la recuperacion de contraseña sigue siendo publica.
- `Dockerfile`: la imagen no cambia.

6.5 Como se elige el proveedor en tiempo de ejecucion

Con anotaciones condicionales de Spring sobre cada implementacion: la de HTTPS se activa cuando la propiedad dice 'resend' y la de SMTP cuando dice 'smtp'. Con eso, en cada arranque existe **un solo bean** de envio.

Por que es mejor fallar al arrancar que enviar en silencio

Si el valor de la propiedad esta mal escrito y ninguna condicion cubre ese caso, Spring simplemente no encuentra el bean y la peticion de recuperacion de contraseña devuelve un error 500 en produccion, sin que nadie se entere hasta que un usuario reporte que no le llega el correo. Con la comprobacion explicita, el despliegue falla al arrancar, que es exactamente cuando se quiere saber.

7. Variables de entorno

7.1 Variables nuevas para Resend

Variable	Obligatoria	Ejemplo	Donde vive
RESEND_API_KEY	Si	re_xxxxxxxxxxxx	Railway (variables del servicio) y application-local.yaml
RESEND_FROM	Si	no-reply@tudominio.com	Idem
RESEND_URL	No	https://api.resend.com	Solo si se quiere dejar explicito en la configuracion
MAIL_PROVIDER	No	resend	Selector de proveedor; si falta, debe existir un default explicito

7.2 Variables actuales y que hacer con ellas

Variable	Hoy	Con Resend recomendado
MAIL_USERNAME	Obligatoria (sin default)	Se puede quitar, pero solo si antes el bloque de correo de application.yaml deja de exigirla. Si se conserva SMTP de respaldo, se queda
MAIL_PASSWORD	Obligatoria (sin default)	Igual que la anterior
MAIL_HOST	Opcional (default smtp.gmail.com)	Solo con SMTP
MAIL_PORT	Opcional (default 587)	Solo con SMTP
FRONTEND_URL	Opcional (default localhost:4200)	Se mantiene y es importante: es la base del enlace del correo

El enlace del correo es lo mas delicado del cambio

El correo lleva *app.frontend-url* mas la ruta de la pantalla de restablecimiento. Si esa variable no apunta al frontend real en el despliegue, el correo **sale bien y no sirve de nada**: el usuario recibe un enlace roto. Es un fallo que no aparece en las pruebas si en local todo funciona, porque en local el valor por defecto ya es correcto. Conviene verificarlo a mano una vez en el despliegue.

7.3 Registros que autentican al remitente

- **SPF**: declara que servidores pueden enviar con tu dominio.
- **DKIM**: firma cada correo para que no pueda alterarse por el camino.
- **DMARC**: dice que hacer con los correos que no pasan las pruebas, y de ahí sale la capacidad de ver los rechazos.
- Los tres se publican como registros DNS en el panel del proveedor de dominio. Resend indica exactamente que valores usar.

8. Paso a paso para aplicarlo

8.1 Antes de tocar el backend

1. Crear la cuenta en Resend y generar la API key.
2. Confirmar que hoy el correo de recuperacion falla en Railway y guardar el log: es la evidencia de que el cambio hacia falta.
3. Crear la **interfaz** y las **dos implementaciones** (estrategia B). En este punto nada cambia todavia: el comportamiento es identico.
4. Anadir la excepcion de entrega y su manejador.

8.2 Probar en local, todavia con SMTP

1. Dejar el proveedor en SMTP y arrancar la aplicacion.
2. Pedir la recuperacion con un correo de pruebas y confirmar que llega.
3. Este paso verifica que la refactorizacion **no rompio nada**, antes de meter una variable nueva. Si falla aqui, el fallo es del refactor y no de Resend.

8.3 Cambiar el proveedor a Resend en local

1. Poner la API key y el remitente en *application-local.yaml*.
2. Empezar con *onboarding@resend.dev* como remitente y usar el correo con el que se creo la cuenta.
3. Cambiar el selector a Resend, reiniciar y volver a pedir la recuperacion.
4. Comprobar el correo recibido y, en el panel de Resend, el estado del envio.

8.4 Verificar el dominio y pasar a produccion

1. Anadir el dominio propio en Resend y publicar los registros DNS.
2. Cambiar el remitente a *no-reply@tudominio.com*.
3. Anadir *RESEND_API_KEY* y *RESEND_FROM*, y comprobar *FRONTEND_URL* en las variables del despliegue.
4. Redeploy y repetir la prueba desde fuera: correo a un Gmail que no es la cuenta del proyecto, para ver la entrega real.

8.5 Mas adelante (opcional)

1. Anadir el respaldo SMTP para el caso de que Resend tenga una incidencia.
2. Meter la cabecera de idempotencia al envio, para que un reintento no genere un correo duplicado. Es la misma idea que ya se aplico a la venta.
3. Registrar los envios con su identificador para poder depurar.

9. Como comprobar que quedo bien

#	Prueba	Resultado esperado
1	Recuperacion con correo existente	Respuesta correcta y correo recibido
2	Recuperacion con correo inexistente	Comportamiento actual: mensaje de usuario inexistente
3	Restablecer con token invalido	Se rechaza con token invalido
4	Restablecer con token vencido (mas de 1 hora)	Se rechaza por expirado
5	Restablecer y entrar con la nueva contraseña	Entra con la nueva y la anterior deja de servir
6	Revisar el contenido del correo	El enlace apunta al dominio real y no a localhost

#	Prueba	Resultado esperado
7	Revisar la carpeta de spam	El correo llega a la bandeja de entrada
8	API key ausente o incorrecta	Falla de forma clara, sin un 500 opaco

La prueba 2 es un problema de seguridad que ya existe

Hoy, si el correo no esta registrado, la respuesta dice explicitamente que no existe una cuenta. Eso permite a alguien **averiguar que correos estan registrados** probando hasta que la respuesta cambia. El uso habitual de este tipo de formulario es devolver siempre la misma respuesta de exito y enviar el correo solo si el usuario existe. **No es parte del cambio a Resend** y conviene tratarlo aparte, pero es el defecto mas serio que hay alrededor de este modulo.

10. Riesgos, limites y decisiones pendientes

10.1 Lo que NO se resuelve con este cambio

- **El fallo hacia arriba.** Si la llamada a la API falla, hoy la excepcion sube y el endpoint devuelve error. Lo correcto para un formulario de recuperacion es registrar el fallo y responder como si nada: el usuario no puede arreglar un problema de red, y darle un error lo invite a insistir.
- **El limite de intentos.** Nada impide pedir recuperacion en bucle y gastar cuota. Es una forma de denegacion de servicio y de correo no deseado. Se resuelve limitando por IP o por correo.
- **El enlace mas el token.** El token es un UUID y dura una hora: es correcto. Solo conviene saber que no hay limite de reintentos por token ni un registro de cuando se pidio cada enlace.
- **El HTML.** Hoy el correo es texto plano. Con Resend se puede enviar *html* sin esfuerzo. Es una mejora opcional, no una necesidad.

10.2 Otras APIs HTTPS por si Resend no convence

Servicio	Plan gratuito	Notas
Resend	~3000 correos/mes	La mas sencilla para developers
Brevo (antes Sendinblue)	300 correos/dia	Buena entregabilidad, interfaz de campanas
SendGrid	100 correos/dia	Muy extendida; pide verificar el remitente
Mailgun	Por consumo con tier	Orientada a correo transaccional
Amazon SES	Muy barato por uso	La mas economica a volumen, mas compleja de configurar

El criterio para elegir entre ellas no es el precio: es cuanto trabajo hace falta hasta que llega el primer correo. En todas se aplica la misma estructura de cambio descrita en la seccion 6, así que decidir despues es barato.

10.3 Las 4 decisiones que conviene tomar antes de escribir

#	Pregunta	Opcion A	Opcion B	Costo de equivocarse
1	Solo Resend, o Resend + SMTP de respaldo?	Solo Resend: menos codigo	Con respaldo: tolera caidas	Bajo. El respaldo se puede anadir despues
2	Dejar SMTP configurado o eliminarlo?	Dejarlo para desarrollo local	Eliminarlo del todo	Medio si se elimina: se pierde la opcion de probar sin terceros
3	Correo en texto plano o con HTML?	Texto plano (como hoy)	HTML con boton	Nulo: es cosmetico
4	Que responde el endpoint si el correo no se pudo enviar?	Subir el error	Registrar y responder exito	Medio: afecta la experiencia del usuario

10.4 Debilidades que quedan

- Dependencia de un tercero: si Resend baja, no hay correo. El respaldo lo cubre, a costa de un poco mas de codigo.
- Los regimenos gratuitos cambian sin avisar. Conviene revisar los limites antes de depender de ellos.
- Una API key en el despliegue es un secreto mas que proteger, con el mismo cuidado que las demas variables sensibles del proyecto.

11. Glosario

Termino	Significado
SMTP	Protocolo con el que un servidor de correo recibe y entrega mensajes
STARTTLS	Cifrado que se negocia dentro de una conexion SMTP ya abierta
API HTTPS	Envio de correo como peticion HTTP en vez de protocolo SMTP

Termino	Significado
API key	Clave que sustituye a usuario y contraseña para llamar a la API
Dominio verificado	Dominio con registros DNS publicados que prueban que es tuyo
SPF / DKIM / DMARC	Los tres registros que autentican al remitente y dicen a los filtros que el correo es legitimo
Idempotencia	Poder repetir la misma peticion sin que el efecto se duplique
Spam	Correo no solicitado; lo que se manda sin verificar el dominio acaba ahi

Resumen final

El problema es de red, no de Java. Cambiar SMTP por una API HTTPS lo resuelve con un plan gratuito y un cambio pequeno y localizado: una interfaz, dos implementaciones y un selector por configuracion. El enlace, el token, la vigencia, el endpoint y el frontend no se tocan. Lo unico que hay que decidir antes es si se quiere un proveedor unico o un primario con respaldo, y eso se puede cambiar despues sin rehacer nada.